Lab 1: Introduction to R, RStudio, and Quarto

Math 141, Week 1

Your name here

Welcome to our first lab assignment! In this lab, you will practice:

1. Warming up with fundamental commands in `R`.
2. Running `R` code from chunks.
3. Viewing and describing data frames in `R`.
4. Rendering a `Quarto` document.

Add any collaborators here!

I collaborated with...

Remember that you are not allowed to use any internet resources or generative AI models (like ChatGPT or Claude) on your assignment. While there are many ways to achieve tasks in R, you may only use packages and functions that have been covered in this class. If you are unsure of how to approach an exercise or of what function to use, help is available to you! Please visit office hours, collaborate with your peers, or post in the Slack channel.

When you have completed Lab 1, follow this submission process:

- **Step 1 (RStudio Server)**: Export the PDF (not the QMD) of your Lab 1 from the RStudio Server. Within the Files tab in the lower right-hand window:
  - Check the box next to "lab01_yourname.pdf".
  - Click on the blue cog and select "Export..." and then "Download."
- **Step 2**: Within Gradescope (which you can access from the course website), click on Lab 01 and select the option "Submit PDF."
  - Then "Select PDF" from where it was downloaded onto your computer and hit "Upload PDF."

- **Step 3**: Within the Question Outline, click on "Exercise 1" and then click on the pages that contain your solutions for Exercise 1. Repeat this process for each Problem. **If you do not assign pages to your submission, you will not receive full credit!**
- **Step 4**: Hit "Submit" and revel in the completion of a task!

First off: Why Quarto?

Quarto is an open-source tool for writing about our data work. We will complete all class assignments in "Quarto Markdown Files", which have a `.qmd` extension.

- Why are we using Quarto (instead of Word, Google Docs, Overleaf, etc)?
  - You can put all your work (code, output, and write-up) in one document which allows for a fully reproducible workflow.
  - When you render the Quarto file, you get a single nicely formatted mixed-media output document. Some of the output options include PDF, HTML, and MS Word. We will output to PDF in STAT 141.
  - You can insert math symbols (with what is called Latex code): μ.
- When editing a Quarto document, you can choose between the Source editor or the Visual editor. I recommend testing out both.
  - **Source editor**: Shows the raw markdown syntax when editing (Megan's personal preference).
  - **Visual editor**: Provides a word processor type interface when editing.

R code and output

To include `R` code and output, you need to create an `R` **chunk**. Here's an example `R` chunk:

```
print("Hello, world!")
```

```
[1] "Hello, world!"
```

- The icons in the upper right-hand corner of an `R` chunk are helpful for running the code in your document.
  - The gray arrow with a horizontal green line tells `R` to run all the chunks above the current chunk.
    - If you click on the button above, nothing happens as this is our first `R` chunk of the document.
  - The green arrow tells `R` to run all the code in the current chunk.

    * Notice what happens when you click on this button.

- Other options for running code in an `R` chunk include:
  - Put the cursor on the same line as the code and hit Command+Enter (for a Mac) and Control+Enter (for Windows).
  - Put the cursor on the same line as the code and hit the "Run" button.
- You can insert an `R` **chunk** into the document by clicking on the green button with a plus sign and C in the panel at the top of the document, or with the shortcut: Command+Alt+i (on Macs) and Control+Alt+i (on Windows), or by typing out the ticks and `{r}` manually.

Exercises

Exercise 1: R Fundamentals

In this problem, we will explore fundamental commands and behavior in R.

a. In R, we use `<-` to assign values to variable names, like `x <- 3`. This saves `x` as an object in our environment, so that we can use it again later. In the R code chunk below, create a variable called `y` with the value 7. (*Note: ticks `---` are used to format text as inline code. Their purpose is purely for formatting text when we're writing about code - when you're actually coding in R inside of chunks, you shouldn't use them.*)

---

b. As discussed above, there are several ways to run code in RStudio. Practice these again on this code chunk, and study how R is performing algebra with the following commands (which may be easier if you run the code line-by-line).

```
x <- 3

### R as a calculator

3 + 3         # We can add numbers in R like this
```

```
[1] 6
```

```r
3 + x
```

```
[1] 6
```

```r
3 - 3          # We can subtract numbers in R like this
```

```
[1] 0
```

```r
x - 3
```

```
[1] 0
```

```r
3*3            # We can multiply numbers in R like this
```

```
[1] 9
```

```r
x*x
```

```
[1] 9
```

```r
30/3           # We can divide numbers in R like this
```

```
[1] 10
```

```r
30/x
```

```
[1] 10
```

```r
3^3            # We can exponentiate numbers in R like this
```

```
[1] 27
```

```r
x^3
```

```
[1] 27
```

```r
(3^3 - 3)*3   # We can use parentheses in R to control the order of operations
```

```
[1] 72
```

```r
(x^x - 3)*x
```

```
[1] 72
```

---

c. The chunk above shows two useful things we can do in R chunks. (1) We can use R like a calculator to do algebra with numbers and/or variables. (2) We can leave ourselves and our collaborators notes, called comments. These start with at least one `#`. Comments aren't executed as code, but they are very helpful for organizing and explaining our work in R. To practice using both of these below, calculate 10 times 5 divided by 2, all subtracted from 30. Practice using comments by adding a commented line below your computation that says the answer to this calculation.

---

d. We've made several edits to this file. You might notice that the file name in the file tab to the upper left ends with a star ("*"), and is bolded a brighter color (maybe red or blue). **This means that your file is not saved, which is dangerous!** To make sure that your work is saved, click the save icon (the light blue floppy disk) right below the file name, or press Command+S (Mac) or Ctrl+S (PC). It is important to get in the habit of saving often to make sure you don't lose your hard work.

---

e. Below, create a new code chunk. You can do this by either typing out the ticks and `{r}` and closing ticks, or use the shortcut Command+Option+I on Mac, or Ctrl+Alt+I on Windows. Inside this chunk, *overwrite* the variable called `x` with the value of 5, and another variable called `z` with the value of `x` multiplied by `y`.

f. Run the chunk below. Where does the answer appear? Find the console pane (usually the bottom left of the RStudio window), and run the same command, `2*50`, in the console. Where does the answer appear now?

```
2*50
```

f. Sometimes, we'll want to do math with many numbers. For example, we may want the sum of a long sequence of numbers, which we won't want to write out using `+`. Luckily, `R` has many built in *functions* that help us do calculations like this. To add 1, 2, and 3, we can use the function `sum()` and write: `sum(1, 2, 3)`. This is equivalent to `1 + 2 + 3`. Create another new chunk below, and use the `sum()` function to add the variables `x`, `y`, and `z` that you created above. *You should not use `+` at all here! `sum()` will add the elements you give it.*

g. It's easy enough to write out `1 + 2 + 3`. Functions like `sum()` really come in handy when we are working with many numbers, usually stored in *vectors*. The chunk below creates a variable called `myvec`, which is a vector containing the numbers 0 through 100. As shown below, we can use the `sum()` function to add up all of the numbers in `myvec`, simply by passing the vector to the `sum()` function. Add a new line at the bottom of this chunk. Using the same idea, use a new function, `mean()`, to take the average of the values in `myvec`.

```
myvec <- 0:100
sum(myvec)
```

```
[1] 5050
```

---

h. By the end of the semester, we will have learned many functions! Every function in `R` has a documentation or "Help" page, which can be useful for remembering what a particular function does, or what syntax it expects. To pull up the "Help" page for a function, we can run `?--the function name--` in the console. For example, try running `?mean` in the console to pull up its help page.

i. Okay, we've made a lot of edits to the file! As you work on your assignments, it's a good idea to **Render** your file into a PDF, which is the format you'll need to submit assignments in. Clicking the **Render** button above tells `R` that you want to compile the contents of the Quarto document into an output document. Find this **Render** button and click it. You may get a message to install some `R` packages. Allow it. You may also get a message about allowing popups. Allow them. Check out the output file. This file is saved in the same directory as your Quarto (qmd) file but with a different extension (pdf).

Notes on Workflow

It can take some time to understand the workflow for interacting with `R` and Quarto documents. That is okay! I recommend the following workflow:

- Make changes to the Quarto document (ex. solve 1-3 subparts of an assignment).
- If the changes involve new `R` code, then run the code (using one of the options tested above) to make sure the code behaves how you are expecting. Keep iterating until you are happy with the code.
- Click "Render" to render the document.
- At some point in the semester (but especially now, when you're starting out), this step will result in an error message! Error messages show up in red in the Console. Your file cannot be rendered if there is an error. When this happens, try not to panic. Debugging is a constant part of coding. Read the error message, and try to find the error and fix it. If you're stuck and not in lab, ask for help on Slack, from your peers, or in office hours.
- If your file renders without an error, look over the output PDF.
  - Is the code displaying how you want?
  - Does the expected output (graphs, tables, etc) appear?
  - Does the output look correct and legible?
- If the output document looks good, start back at the top and make new changes. If not, go back to your code and problem-solve.

Exercise 2: The R Environment

a. Find the environment pane in your RStudio window (usually in the upper right). What do you see there?

b. In the environment pane, click the little broom icon in the upper bar. Click yes on the window that opens. What happened to the environment? What happens if you now try to run your chunks from Exercise 1 step (f)?

c. Now is a good time to see what happens when you want to render your file, but there is an error in your code. Try hitting "Render." Find the error message in the console. What is it telling you? How does this relate to what's going on with the environment?

d. This kind of error is very common. Let's fix it for our case. Find the "Run" buttons at the upper right corner of this chunk, and select "Run all chunks above." What happened to the environment?

e. Try clicking "Render" again. Did it work this time?

f. Let's practice using exponents again to create another variable, `s`. Run the chunk below one line at a time using Command+Enter (for a Mac) or Control+Enter (for a PC), and write out the value associated with each line below. Why does each line affect the environment differently? Where did you find the values?

```
s <- 7^7

7^5
```

---

g. Finally, let's practice making a vector, and creating **character** data types. In R, we can create vectors using the function `c()`. For example, try running `c(1, 2, 3)` in the console. All elements of a vector must have the same data type (ex. all numeric, or all character). We create characters by writing `"some text"`. In a new chunk below, create a vector called `a` with three character elements, with each element representing one of your favorite genres of music. What information about this object is stored in the environment?

R Packages

- **Useful definition**: People refer to the `R` session that loads without any add-ons as **base R**.
- Base `R` contains lots of useful commands to analyze data, as well as some data sets. However, people have developed new commands and new data sets which are not packaged in this standard version of the program. Therefore, people create **packages** which contain these new objects.
- Whenever we want to use the commands or load the data from a package, we need to load the package itself first.
- Below is the code that loads the `openintro` package in your Quarto document (which we'll use in the final exercise of the lab). You will need to run the code (as described above) if you want `openintro` to be loaded in your RStudio Session.

```
library(openintro)
```

- Go to the **Packages** tab in the lower right. Notice that `openintro` is listed and if you ran the code above, then it is loaded so the box next to `openintro` should be checked.
- You only need to install a package once, but you need to load a package (using the `library()` function) every time you want to use the commands in the package. If you are using RStudio Server, almost all of the packages that you'll need should already be installed.

Exercise 3: Exploring Data Frames

In this problem, we are going to explore an existing data frame in `R`.

a. For data frames in general, what does a row represent? What does a column represent?

---

b. Run the following code to grab the `yrbss` dataset, which is contained in the `openintro` package that we already loaded.

```
data(yrbss)
```

c. Run the following, and begin exploring the data called `yrbss`. (Leave in the `eval: false`.) Some useful functions to get started include: `head()` and `str()`. You don't need to write any written response for this part, just use this space to explore the data set and answer the following questions.

Pro tip: For datasets in packages, we can also run `?` followed by the dataset name to open up the documentation file in the Help pane that describes the dataset and provides information on the variables.

```
View(yrbss)
?yrbss
```

d. What does a row of the data frame represent? How many rows are in the data frame?

---

e. What does a column of the data frame represent? How many columns are in the data frame?

f. Is `age` recorded as a categorical or a quantitative variable? How about `hours_tv_per_school_day`? Explain your reasoning.

g. We will learn to be careful with how we treat missing values, which are typically labeled as `NA` in the date frame. What are two possible reasons why the the variable `text_while_driving_30d` might be recorded as `NA`?

h. One way to look at or work with a single variable is by using `$` to extract it. Uncomment the first line and run the chunk below, which extracts just the `age` column as a vector, and takes its average. When you're done checking it out, re-comment the first line so that your PDF doesn't include all 13,000+ ages!

```
# yrbss$age          # This command extracts age
mean(yrbss$age, na.rm = TRUE)   # We can combine this extraction with a function
```

```
[1] 16.15704
```

i. Render your file, and carefully check your PDF one last time before following the instructions for submission at the beginning of the lab. As we move forward in the class with assignments, you are responsible for making sure to check that the formatting and content of your PDF reflects the work in your `.qmd` file.